DRIFTLINE

UNLEARNING NOTE / PROTOTYPE 01

DIRECT > GENERATE > FALSIFY > DECIDE > RECORD

What I unlearned *while building Driftline*

39/39	44.34 ms	5 DAYS
TESTS PASS	10K MEDIAN	LOGGED

The largest change was not that I wrote less code. It was that I stopped treating authorship - or an AI's confidence - as evidence that the work was right.

01 / MAKE THE CLAIM FALSIFIABLE

My default habit is to begin once the happy path sounds plausible and postpone awkward cases until the interface feels real. For Driftline, I reversed that sequence. Before implementation, I made **stalled**, **priority**, and **done** falsifiable. I asked AI to attack the rules, compare alternatives, and generate hostile data. Then I checked those proposals with hand calculations, invariants, tests, and browser behavior. AI widened the search; it did not get to declare its own output correct.

02 / AI-FIRST DOES NOT REQUIRE RUNTIME AI

Driftline's runtime is deterministic and uses no external model or API. AI helped shape the schema, generate edge cases, draft implementation, and critique the result. Keeping it out of the runtime made the user-facing decision reproducible: the same CSV, as-of date, and base limit produce the same queue. In hiring, fluent generated language can make a weak inference sound authoritative, so each explanation comes from validated facts and a fixed template.

03 / A RED TEST PROVES DISAGREEMENT

My first fixed-sample test failed because I expected seven distinct owners while the program reported six. It was tempting to fix the code. Inspecting the nine flagged rows showed the implementation was right: repeated owners had been double-counted in my mental tally. I corrected the oracle, not the algorithm, and kept the failed test in the log. Expected values deserve review too.

04 / CORRECT ARITHMETIC IS NOT ENOUGH

The browser exposed a different class of failure. The sentence 'against a 11-day limit' passed every arithmetic check and still damaged credibility. A common local port was also captured by an old service worker from another app, so a successful HTTP response did not prove the intended page was visible. I fixed the wording, moved the server to port 8765, and verified the actual interface.

05 / RAISE THE BURDEN OF PROOF

I used to associate difficulty with more features. Here the harder move was to make claims harder to get wrong: stage-aware limits, multi-stage boundaries, contradictory-date quarantine, stable ties, identity-field invariance, workload ceilings, fail-closed headers, review-labeled exports, sensitivity analysis, and a 10,000-row benchmark. I cut ATS sync, automatic messaging, persistence, and production claims. A responsive audit exercised 390, 500, 821, 1180, and 1440 CSS pixels; that is not a physical-device or full accessibility claim.

06 / AUTOMATION CAN KEEP THE HUMAN

Driftline organizes attention; it does not judge people, send a follow-up, or change a hiring record. Names identify rows but do not affect priority. Unsafe structure stops ranking. Process debt describes workflow backlog, not candidate worth or recruiter performance. Every exported row carries NEEDS_HUMAN_REVIEW and WORKFLOW_DELAY_ONLY so the boundary survives outside the interface.

07 / HONEST SPEED

The review found that copied evidence had drifted across the note, README, report, and video. The Day-5 canonical snapshot records **39/39** tests passing and 10,000 rows at **44.34 ms median / 57.80 ms p95**, correctness PASS, with outliers retained. All five dates are logged: Day 1 contains the build, Days 2-4 record no work, and Day 5 contains final verification. Preserving the gap matters more than manufacturing iteration, and none of these measurements makes the prototype production-ready.